Module 1 Lab Session 2: Query and Classification

Field	Value
Module	M1 C# 14 Essentials
Exercises	4 (Guard Clauses & Pattern Matching), 5 (Collections & LINQ)

Prerequisites Check Before You Start

Your Models.cs must contain all types from Session 1: EnrollmentRecord, Course, Student, and IGradable (with Quiz and LabAssignment). If any are missing, go back to Session 1 and add them the exercises below will not compile without them.

Exercise 4: Defeating the "Pyramid of Doom" (LO 1.6: Pattern Matching & Guards)

The situation: The enrollment team asked for a simple validation: "Before registering a student, check that the student exists, the course exists, and the course is not full." The previous developer wrote this:

```
if (student != null) {
   if (course != null) {
      if (course.Capacity > 0) {
         // Success  buried three levels deep
      }
   }
}
```

This nested if structure is called the Pyramid of Doom. At 2 AM when production is down, the engineer reading this code cannot tell at a glance what the preconditions are. Every new condition adds another nesting level. By the time you have five checks, the actual logic is indented so far to the right it scrolls off the screen. Guard clauses solve this you check each precondition at the top and exit immediately if it fails. The happy path stays flat and readable.

Decision rule (use this in production code):

- Use guard clauses for required preconditions (null checks, invalid ranges, impossible states). Throw immediately.
- Use switch expressions for clear classification logic (status mapping, score bands, risk levels).

Trade-off you should know: Guard clauses make the happy path easy to read, but you must decide when to throw versus when to return a domain result. Throw for programmer errors and violated preconditions. Return a domain result (like an error enum or a result object) when the case is expected in normal workflow.

Step 1 Build the Enrollment Service

Create a new file called EnrollmentService.cs:

```
public class EnrollmentService
{
    public EnrollmentRecord ProcessRegistration(Student? student, Course? course)
    {
        // TODO 1: Add guard clauses  fail fast if student is null, course is null,
        //       or course capacity is zero or negative.
        //       Use ArgumentNullException for nulls, InvalidOperationException for full course.
        // Stuck? Pattern: if (param is null) throw new ArgumentNullException(nameof(param));

        // TODO 2: Use a switch expression on student.GPA to classify academic standing:
        //       >= 3.5  → "Honors"
        //       >= 2.5  → "Good Standing"
        //       < 2.5   → "Academic Warning"
        //       Print the result: $"{student.Name} is in {standing}."
        // Stuck? Pattern: string result = value switch { >= X => "Label", ... };

        // TODO 3: Return a new EnrollmentRecord with student.Id, course.Code,
        //       and DateTime.UtcNow.
    }
}
```

Step 2 Test It

In Program.cs, add:

```
var service = new EnrollmentService();

// Test 1: Valid registration
var validStudent = new Student { Id = "S1", Name = "Abeba", Age = 20, GPA = 3.8m };
var validCourse = new Course { Code = "CS-401", Title = "Advanced C#", Capacity = 30 };
var result = service.ProcessRegistration(validStudent, validCourse);
Console.WriteLine($"Enrolled: {result.StudentId} in {result.CourseCode}");

// Test 2: Null student  should throw
try
{
    service.ProcessRegistration(null, validCourse);
}
catch (ArgumentNullException ex)
```

```
{
    Console.WriteLine($"Guard caught: {ex.ParamName}");
}

// Test 3: Full course  should throw
var fullCourse = new Course { Code = "CS-402", Title = "Full Course", Capacity = 1 };
fullCourse.EnrolledCount = 1;
try
{
    service.ProcessRegistration(validStudent, fullCourse);
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"Business rule: {ex.Message}");
}
```

Run it:

dotnet run

What you should see:

- Enrolled: S1 in CS-401 for a valid registration
- Guard caught: student when passing null
- A standing classification like Abeba is in Honors.
- Business rule: ... when the course is full

Troubleshooting:

Problem	Likely cause	Fix
Missing return compiler says “not all code paths return a value”	You did not add the return statement after the guard clauses	Add return new EnrollmentRecord(student.Id, course.Code, DateTime.UtcNow); at the end
Switch expression warning “not all cases covered”	Missing a fallback arm	Add _ => "Academic Warning" as the final case
Wrong exception type for full course	Using ArgumentException instead of InvalidOperationException	Full course is a runtime state, not a bad argument use InvalidOperationException
EnrollmentRecord constructor error	Wrong parameter order	Constructor expects (string StudentId, string CourseCode, DateTime EnrolledAt)

Exercise 5: The Analytics Dashboard (LO 1.5: Collections & LINQ)

The situation: The Head of Faculty needs a leaderboard report by end of day. She wants: all Honors students sorted by GPA, the class average, and a breakdown of students by academic standing. The data is in a list. You have LINQ.

From this exercise forward, the solution code is not fully provided for every step. You will implement logic using the TODO comments as guidance the same way a senior developer leaves code review comments for a junior. If you get stuck, each TODO has a "Stuck?" hint.

Step 1 Create the Student Data

In Program.cs, add:

```
// C# 12+ Collection Expressions  the modern way to initialize lists
List<Student> students = [
    new Student { Id = "S1", Name = "Abeba", Age = 22, GPA = 3.8m },
    new Student { Id = "S2", Name = "Kidane", Age = 21, GPA = 2.4m },
    new Student { Id = "S3", Name = "Dawit", Age = 20, GPA = 3.1m },
    new Student { Id = "S4", Name = "Sara", Age = 23, GPA = 3.9m },
    new Student { Id = "S5", Name = "Frehiwot", Age = 19, GPA = 2.0m },
    new Student { Id = "S6", Name = "Yonas", Age = 24, GPA = 3.5m },
    new Student { Id = "S7", Name = "Meron", Age = 22, GPA = 1.8m },
    new Student { Id = "S8", Name = "Tesfaye", Age = 21, GPA = 2.9m }
];
```

Step 2 Build the Honors Leaderboard

```
var leaderboard = students
    // TODO 1: Extract students where GPA is >= 3.5m
    // TODO 2: Sort the remaining students by GPA descending
    // TODO 3: Project the result so we only keep the 'Name' string
    // TODO 4: Materialize the lazy query into a concrete List
    ;

Console.WriteLine($"Found {leaderboard.Count} Honors Students:");
foreach (var name in leaderboard)
{
    Console.WriteLine($"- {name}");
}
```

Decision rule (use this in real projects):

- Use LINQ when you need readable data transformations (filter, sort, project, aggregate).
- Materialize with .ToList() only when you need a concrete snapshot now.

Trade-off you should know: LINQ is usually clearer and less error-prone than manual loops, but deferred execution can surprise you if the source collection changes before enumeration. Call .ToList() when you need the results locked in.

Step 3 Class Average

```
// TODO 5: Use LINQ to calculate the average GPA across all students.
//       Format it to 2 decimal places using :F2.
decimal averageGpa = ____
// Stuck? Pattern: students.Average(s => s.SomeProperty)

Console.WriteLine($"\nClass Average GPA: {averageGpa:F2}");
```

Step 4 Group by Academic Standing

```
// TODO 6: Use .GroupBy with a switch expression to classify each student.
//       GPA >= 3.5 → "Honors", >= 2.5 → "Good Standing",
//       >= 2.0 → "Probation", < 2.0 → "Academic Warning"

var standingGroups = _____
// Stuck? Pattern: .GroupBy(s => s.GPA switch { >= X => "Label", ... })


Console.WriteLine("\n--- Academic Standing Report ---");
foreach (var group in standingGroups)
{
   Console.WriteLine($"\n{group.Key} ({group.Count()}):");
   foreach (var s in group)
   {
      Console.WriteLine($"  {s.Name}  GPA: {s.GPA}");
   }
}
```

Step 5 Collection Expressions with Spread

```
// TODO 7: Use the spread operator (..) to merge two arrays and append a value.
// Stuck? Pattern: string[] combined = [..array1, ..array2, "extra"];
string[] backendCourses = ["C#", "ASP.NET Core"];
string[] frontendCourses = ["TypeScript", "Angular"];
string[] allCourses = ___ // TODO
Console.WriteLine($"\nFull curriculum: {string.Join(", ", allCourses)}");
```

Step 6 Run and Verify

dotnet run

What you should see:

- Found 3 Honors Students: with Sara, Abeba, and Yonas (in GPA descending order: 3.9, 3.8, 3.5).
- Class Average GPA: 2.93

- Academic standing groups: Honors (3), Good Standing (2), Probation (2), Academic Warning (1).
- Full curriculum: C#, ASP.NET Core, TypeScript, Angular, Capstone

Troubleshooting:

Problem	Likely cause	Fix
Wrong order in leaderboard	.OrderByDescending placed after .Select	Chain order: .Where() → .OrderByDescending() → .Select() → .ToList()
Compile error on leaderboard.Count	Pipeline does not end with .ToList()	IEnumerable has .Count() (method), List has .Count (property). Add .ToList()
DivideByZeroException on average	Empty student list	Guard with if (students.Count > 0) before calling .Average()
Spread syntax .. red underline	Older C# version	Requires C# 12+. Add <LangVersion>latest</LangVersion> to csproj if needed
Groups appear in unexpected order	GroupBy preserves encounter order, not alphabetical	This is correct behavior groups appear in the order of first occurrence

Session 2 Checkpoint

Before moving to Session 3, confirm all three:

- ☐ ProcessRegistration(null, course) throws ArgumentNullException with the parameter name student
- ☐ The honors leaderboard prints Sara, Abeba, Yonas in that order (GPA descending: 3.9, 3.8, 3.5)
- ☐ The academic standing report shows four groups with correct counts: Honors (3), Good Standing (2), Probation (2), Academic Warning (1)